

DAY 1 • LECTURE 2 • 45 MINUTES

Into the Deep

From Containers to Kubernetes Pods

Your raft is built. Now we launch it into open water.

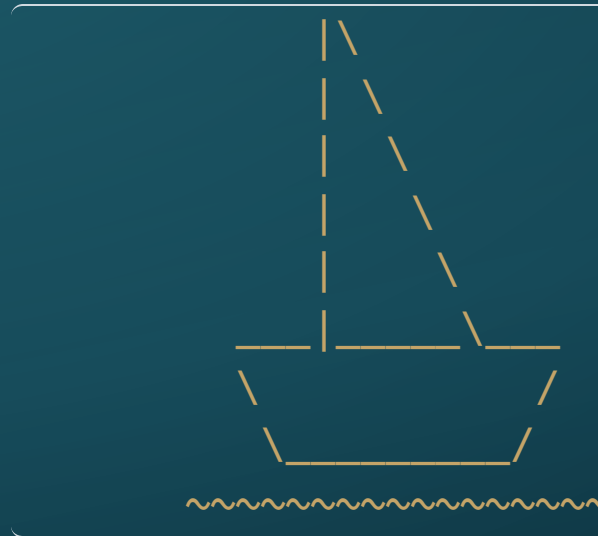


Where we are

- ♦ Lab 01 is done — you have a container **image** sitting in Harbor.
- ♦ That's a raft, built and beached. It doesn't sail itself.
- ♦ The next 45 minutes: how Kubernetes takes that image and puts it to sea.
- ♦ Four short legs:
 - Why one container isn't a fleet
 - The **Pod** — what Kubernetes actually runs
 - Declaring what you want, instead of hand-steering
 - Why Pods are built to be thrown away
- ♦ Then a quick flash poll, and **Lab 02 — Paddling Out.**

PART I

Why a Raft Isn't a Fleet



"One raft is a tale. A fleet is a future. Ye can't crew the whole ocean alone." — the Boatswain

One container, one problem

- ♦ On your VM you can already do this: `docker run my-raft`
- ♦ It works — until it doesn't:
 - the container **crashes** and stays down
 - the **host reboots** and nothing comes back
 - you need **ten** copies, not one
 - the machine it runs on fails, and it has to move
- ♦ A lone container has nothing watching it. Something has to — and "a person doing it by hand" is not a plan that survives contact with reality.

You need a harbour-master

- ♦ Something that watches every container and **keeps the fleet sailing**:
 - restarts the ones that sink
 - schedules them across whatever machines you have
 - scales them up and down on demand
 - replaces them when a whole machine is lost
- ♦ That something is an **orchestrator**.

Meet the orchestrator: Kubernetes

- ♦ **Kubernetes** is the orchestrator the island runs on — the book calls it **Captain Kube**.
- ♦ You hand Captain Kube your containers and a description of what you want.
- ♦ It makes the ocean **match that description** — and keeps it matching.
- ♦ *We meet Captain Kube properly tomorrow morning.* Today: just enough to paddle out.

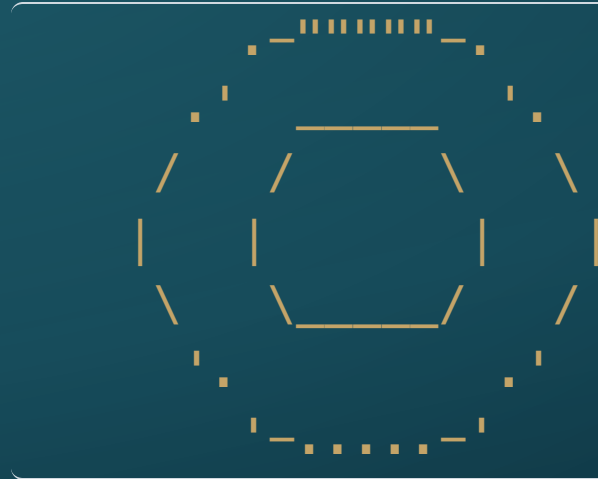
The real shift: telling vs. asking

- ♦ **Docker** is *imperative* — "run this container, right now." A one-time order.
- ♦ **Kubernetes** is *declarative* — "I want one of these to **always** exist."
- ♦ You describe the **destination**. The cluster does the steering, continuously.
- ♦ Pull a Pod out from under it, and Kubernetes notices and acts.

Stop hand-steering every wave. Hand over the chart and let the crew hold course.

PART II

A Life-Jacket Called a Pod



"Never send a crate to sea without a life-jacket. The deep don't forgive." — the Boatswain

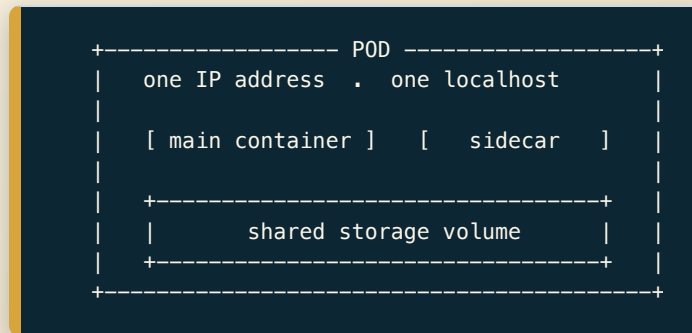
Kubernetes doesn't run containers

- ◆ Here's the surprise: the smallest thing Kubernetes runs is **not** a container.
- ◆ It's a **Pod**.
- ◆ A raw container is just a box. Kubernetes wraps that box in a life-jacket — and the life-jacket is what it actually schedules, watches, and restarts.

So what is a Pod?

- ♦ A **Pod** is the smallest deployable unit in Kubernetes.
- ♦ It wraps **one — or more — containers** that are always:
 - **scheduled together**, on the same machine
 - **started and stopped together**, sharing one lifecycle
 - **networked together**, sharing one address
- ♦ Most Pods hold exactly **one** container. But the wrapper can hold more — and that turns out to be useful.

Why wrap it? Containers in a Pod share



- ♦ One **IP address** for the whole Pod — containers inside reach each other on `localhost`.
- ♦ Shared **storage** — they can hand files back and forth.
- ♦ One **lifecycle** — born together, die together.

The sidecar pattern

- ♦ Most Pods are one container. But you can add a **helper** alongside it:
 - a **log shipper** that forwards your app's logs
 - a **proxy** that handles encryption or routing
 - a **file-syncer** that pulls in fresh content
- ♦ Main container + sidecar, sharing one Pod's network and storage.
- ♦ The app stays simple; the helper does the plumbing — **without changing your image.**

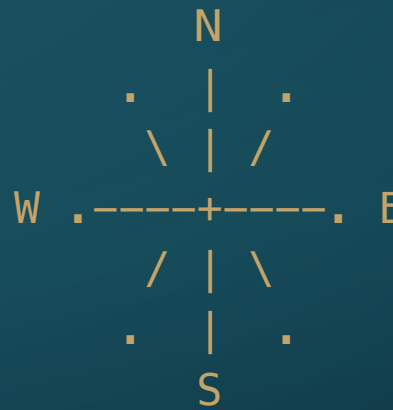
Pod vs. Container

	Container	Pod
What it is	a packaged process	Kubernetes' smallest unit
Holds	one application	one <i>or more</i> containers
Network	(on its own)	one shared IP address
You...	build it (Lab 01)	deploy it (Lab 02)

You build containers. You deploy Pods. The Pod is the box Kubernetes can actually carry.

PART III

Charts, Not Hand-Steering



"Hand the sea a good chart and it'll hold your course while ye sleep." — the Boatswain

Kubernetes objects are written down

- ♦ Everything in Kubernetes — a Pod included — is described in a **manifest**: a **YAML** file.
- ♦ YAML is just structured text: keys, values, indentation.
- ♦ A Pod manifest can run 30+ lines, and it **looks** intimidating.
- ♦ Good news, and the rule for this whole seminar:

You do not write Kubernetes YAML from memory. You generate it.

Generate, don't memorise

```
kubectrl run my-raft \  
  --image=harbor.wagbiz.org/raft-fleet/<your-name>:v1 \  
  --dry-run=client -o yaml > pod.yaml
```

- ♦ `--dry-run=client` — *"build the manifest, but don't create anything yet."*
- ♦ `-o yaml` — print it as YAML.
- ♦ `> pod.yaml` — save it to a file you can open and read.
- ♦ Kubernetes just wrote your manifest **for** you.

The workflow: generate, read, apply

1. GENERATE		2. READ / EDIT		3. APPLY
-----		-----		-----
kubectl run	-->	open pod.yaml	-->	kubectl apply
--dry-run=client		plain, readable		-f pod.yaml
-o yaml > pod.yaml		YAML you can tweak		=> Pod is live

- ♦ You never face a blank file. You start from a working draft.
- ♦ Read it, adjust it if you need to, then **apply** it to the cluster.

Claim your territory: Namespaces

- ♦ One cluster. Around **thirty** sailors sharing it.
- ♦ A **Namespace** is your own patch of ocean — your work, walled off from everyone else's.

```
kubectl create namespace <your-name>  
kubectl config set-context --current --namespace=<your-name>
```

- ♦ The second line says "*always work in my patch*" — so you stop typing `-n` every time.
- ♦ Your `my-raft` Pod and your neighbour's never collide.

Seeing the whole ocean

- ♦ `kubectl get pods` — the Pods in **your** namespace.
- ♦ `kubectl get pods -A` — **every** Pod, in **every** namespace, across the whole cluster.
- ♦ That `-A` flag is how you see past your own patch of water.
- ♦ Keep it in your back pocket — Lab 02 ends with a reason to need it.

PART IV

Ghost Ships

```
  .-~::~~::~-.  
 / (o) (o) \  
 \      <    /  
  .-'~::~~::~'-.  
 / / / | \ \ \ \  
 ( ( ( | ) ) ) )  
 \ \ \ | / / / /
```

*"You best start believing in ghost stories, Miss Turner... you're in one." — Captain Barbossa,
Pirates of the Caribbean: The Curse of the Black Pearl*

Cattle, not pets

- ♦ A Pod is **disposable** — by design.
- ♦ It can crash, be rescheduled to another machine, or be replaced at any moment.
- ♦ That is not a failure. That is Kubernetes **keeping the chart matched**.
- ♦ So: don't name it, don't nurse it, don't hand-tune it. **Don't grow attached.**

The Ghost Ship

```
kubectl exec into a running Pod
|
+-- hand-edit a file inside it      (looks fine!)
|
kubectl delete pod + kubectl apply -f pod.yaml
|
v
the hand-edit is GONE - rebuilt from the immutable image
```

- ◆ Changes you make **inside** a running Pod live only in that Pod.
- ◆ Delete it, recreate it from `pod.yaml`, and those changes **vanish**.

*If it isn't baked into the **image** or written in the **manifest**, it will sink.*

Your diagnostic spyglass

When a Pod misbehaves, three commands tell you almost everything:

```
kubectl get pods      is it running? what's its status?  
kubectl logs <pod>    what did the container print?  
kubectl exec -it <pod> -- sh  board the Pod and look around
```

- ♦ `get` for the **status**, `logs` for the **story**, `exec` to **walk the deck**.
- ♦ You'll use all three in Lab 02 — and lean on them all week.

Instructor Superpower

ONE CLUSTER, THIRTY SAFE SANDBOXES

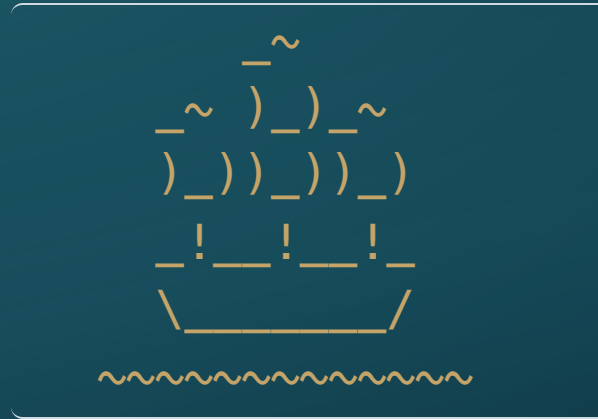
- ♦ Every student's work is two things you can **see**: a readable **manifest** and an isolated **namespace**.
- ♦ A student stuck? `delete` the Pod, re-`apply` the file — a clean reset in seconds.
- ♦ The real shift: the environments you can *choose* to support have radically expanded. You don't wait for IT to say "sure, we can support that" — you build it yourself, for a lesson, a project, a whole course.

Up next: Flash Poll, then Lab 02

- ♦ **Flash poll** — five quick questions to check the crew before we sail. Open `poll.wagbiz.org`.
- ♦ Then **Lab 02 — Paddling Out**:
 - board the live cluster with your Rancher kubeconfig
 - claim your **namespace**
 - **generate** `pod.yaml`, then `apply` it
 - practice the spyglass: `logs` and `exec`

Outcome: your raft afloat in the live cluster — and you, able to tell why when it isn't.

Paddle out.



The raft is yours. The ocean is the cluster. Go.

